CODE AND BRANCH COVERAGE REPORT

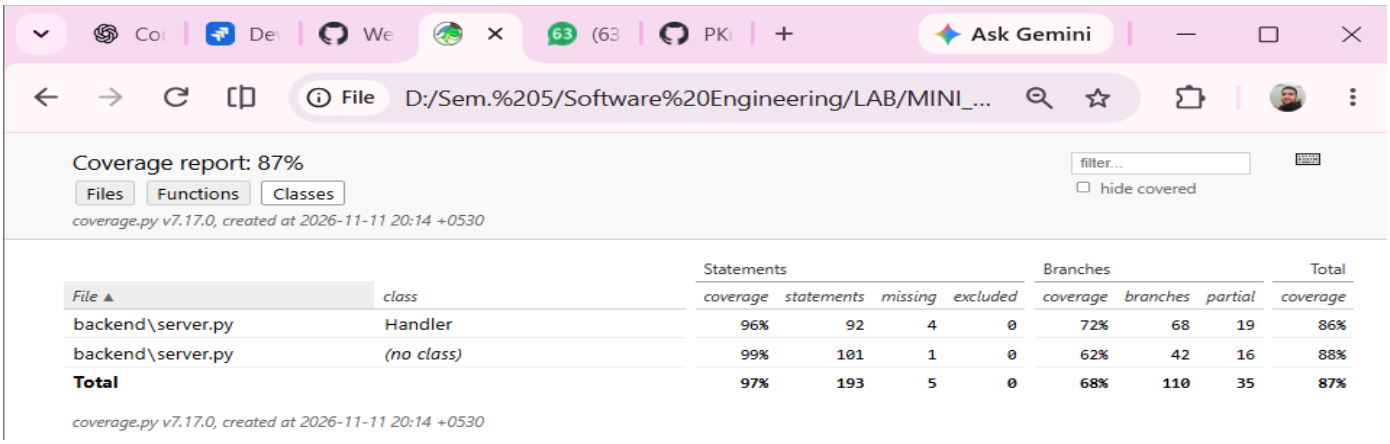
Project: Web Based ML Model Evaluator

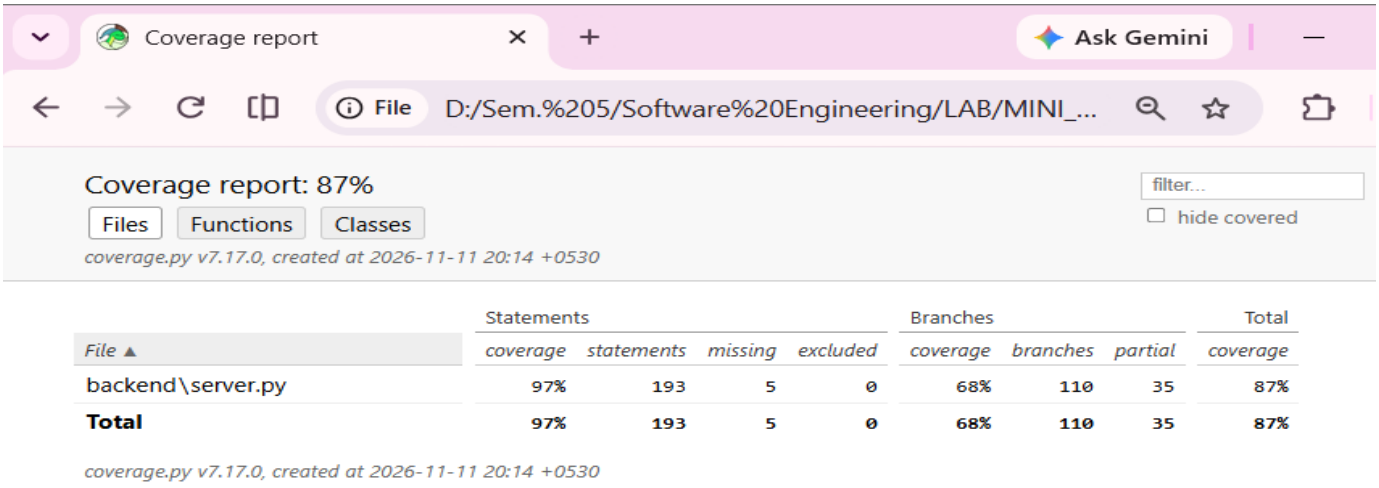
Authors:

PES2UG24CS368	–	PRATHAM KUMAR	- prathamkumar041104@gmail.com
PES2UG24CS324	–	NIRUPAMA JAYARAMAN	- jnirupama06@gmail.com
PES2UG25CS824	–	SHREYAS BELLAD	- shreyasbellad2006@gmail.com
PES2UG24CS339	–	PARINITHA ACHARYA	- parinitha.acharya2006@gmail.com

Approvals

Role	Name	Signature / Email	Date
Course Coordinator	Sheela Devi M.	sheelam@pes.edu	—





1. INTRODUCTION

1.1 Project Overview

The Web-Based ML Model Evaluator (ModelLab) is a web-based application designed to simplify and streamline the process of developing, training, and evaluating machine learning models. The platform provides an integrated environment where users can upload datasets, explore data, perform preprocessing operations, configure machine learning experiments, train models, and evaluate their performance through appropriate evaluation metrics.

The primary objective of ModelLab is to provide users with a centralized platform for experimenting with different machine learning algorithms without requiring them to independently implement the entire model development and evaluation workflow. The application supports both classification and regression tasks and integrates frontend, backend, database, and machine learning processing components.

Since ModelLab contains multiple functions, processing operations, and conditional execution paths, it is important to evaluate not only whether the implemented code executes successfully but also how much of the source code and its decision-making logic is exercised during testing. Code coverage and branch coverage analysis are therefore performed to assess the extent to which the implemented test suite exercises the application's source code.

1.2 Introduction to Code Coverage and Branch Coverage

Code coverage is a software testing metric used to measure the proportion of source code executed during the execution of a test suite. It helps identify portions of the application that are executed by the existing test cases and highlights code that may not have been adequately tested.
In ModelLab, code coverage analysis is used to determine how extensively the automated test suite executes the implemented application logic.

Branch Coverage

Branch coverage is a software testing metric that measures whether the possible outcomes of decision-making statements in a program have been executed. These decisions include conditional statements such as if, else, and other control-flow conditions.
For example, when a program contains an if-else condition, branch coverage examines whether the test cases execute both the true and false outcomes of that decision.
In ModelLab, branch coverage analysis helps evaluate whether the test cases exercise different conditional execution paths, including valid inputs, invalid inputs, successful operations, and rejected operations.

Difference Between Code Coverage and Branch Coverage

Code coverage provides an indication of how much of the source code is executed, whereas branch coverage focuses specifically on whether the different outcomes of conditional decisions are exercised. Both metrics are useful for identifying gaps in the existing test suite.

1.3 Code Coverage and Branch Coverage Strategy and Implementation

A structured coverage analysis strategy was adopted to evaluate the effectiveness of the automated testing implementation developed for ModelLab.

The existing automated test suite was used as the basis for coverage measurement. The test cases were executed while collecting information about the source-code statements and conditional branches exercised during execution.

The following strategy was adopted:

1. **Test Suite Preparation:** The existing automated test modules were organized to cover the major functionalities and processing operations of ModelLab.
2. **Coverage Instrumentation:** Coverage measurement was performed during test execution to collect information about executed statements and conditional branches.
3. **Statement Coverage Analysis:** The executed and unexecuted source-code statements were examined to determine the overall statement coverage percentage.
4. **Branch Coverage Analysis:** Conditional execution paths were examined to identify branches exercised by the test cases and branches that remained partially or completely untested.
5. **Report Generation:** Coverage results were generated in machine-readable and HTML formats to provide both a summarized overview and detailed source-code coverage information.
6. **Result Analysis:** The generated reports were examined to identify coverage percentages and areas where additional test cases could improve the testing effectiveness.

The coverage analysis was performed alongside the automated testing process. The generated reports provide an overview of statement coverage, branch coverage, and detailed coverage information for the application's source code.

The analysis helps evaluate the effectiveness of the existing test suite and identify opportunities to improve the overall testing completeness of ModelLab.

2. Coverage Implementation and Analysis

For this section, we will document how coverage measurement was incorporated into the existing ModelLab testing process, how statement and branch coverage were collected, and which reports were generated.

One important implementation detail is now confirmed by the recorded execution output: ModelLab's coverage analysis was performed using Python's coverage.py tool with branch measurement enabled. The report

measured the backend source file backend/server.py while executing the automated test suite. The recorded output identifies 193 statements and 110 branches.

2.1 Coverage Measurement Strategy

The coverage analysis was carried out using the existing automated test suite rather than creating a separate application implementation. The consolidated execution script, run_all_tests.py, runs the five testing categories:

1. Unit Testing – 32 test cases.
2. Functional / Component Testing – 15 test cases.
3. Integration Testing – 6 test cases.
4. System Testing – 4 test cases.
5. End-to-End Testing – 2 test cases.

This results in 59 automated test cases being executed during the recorded coverage run.
The coverage output focuses on backend/server.py and records the number of executable statements, missed statements, branches, and partially covered branches. This allows the analysis to distinguish between code that was executed and conditional paths that were not fully exercised.

2.2 Coverage Implementation

The coverage analysis was performed using the Python coverage.py library. The tool was used to collect execution information from the ModelLab backend while the automated test suite was running.
The implementation focused on the following activities:

1. Source Code Measurement: The backend module backend/server.py was selected for coverage measurement.
2. Statement Tracking: The coverage tool recorded which executable statements were reached during test execution.
3. Branch Tracking: Branch measurement was enabled to identify whether alternative outcomes of conditional statements were exercised.
4. Coverage Summary: The collected information was presented in a tabular format containing statement count, missed statements, branch count, partially covered branches, and coverage percentage.
5. Report Generation: The execution process generated HTML and JSON reports for further inspection and analysis.

The consolidated testing script, run_all_tests.py, was used to execute the complete testing process and display the final coverage summary.

```
def main():
    started = datetime.now(timezone.utc)
    cov = coverage.Coverage(source=[str(ROOT / "backend")], branch=True, data_file=str(ROOT / ".coverage"))
    category_results = []
    all_output = []
    print("=" * 76)
    print("MODELLAB â€" COMPLETE SOFTWARE TESTING")
    print("=" * 76)
    print(f"Project directory: {ROOT}")
    print(f"Started: {started.astimezone().isoformat(timespec='seconds')}")
```

Figure 2.1: Initialization and Configuration of Code and Branch Coverage.

Report: The coverage analysis is initialized using the Python coverage.py library. The backend directory is specified as the source for measurement, and branch coverage is enabled through the branch=True parameter. The coverage data is stored in the .coverage file for subsequent report generation.

```
    cov.start()

    try:
        for label, module_name in CATEGORIES:
            print(f"\n{'=' * 20} {label} {'=' * 20}")
            module = importlib.import_module(module_name)
            suite = unittest.defaultTestLoader.loadTestsFromModule(module)
            stream = StringIO()
            result = unittest.TextTestRunner(stream=stream, verbosity=2).run(suite)
            output = stream.getvalue()
            print(output, end="" if output.endswith("\n") else "\n")
            all_output.append(f"{label}\n{output}")
            status = "PASSED" if result.wasSuccessful() else "FAILED"
            count = result.testsRun
            category_results.append({
                "category": label, "module": module_name, "status": status,
                "tests": count, "failures": len(result.failures),
                "errors": len(result.errors),
                "details": [str(x[0]) + ": " + x[1] for x in result.failures + result.errors]
            })
            print(f"--> {label}: {status}; {count} tests; "
                  f"{len(result.failures)} failures; {len(result.errors)} errors")
    finally:
        cov.stop()
        cov.save()
```

Figure 2.2: Execution of Automated Test Cases and Collection of Coverage Data.

Report: Coverage measurement begins before the automated test execution using cov.start(). The script sequentially loads and executes the five testing categories using Python's unittest framework. After execution, cov.stop() terminates coverage collection and cov.save() stores the collected execution data.

```
print("\n" + "=" * 76 + "\nSTATEMENT AND BRANCH COVERAGE\n" + "=" * 76)
coverage_stream = StringIO()
try:
    cov.report(file=coverage_stream, show_missing=True)
    cov.html_report(directory=str(REPORTS / "coverage_html"))
    cov.json_report(outfile=str(REPORTS / "coverage.json"))
    coverage_ok = True
except Exception as exc:
    coverage_stream.write(f"Coverage report generation failed: {exc}\n")
    coverage_ok = False
print(coverage_stream.getvalue())
all_output.append("COVERAGE\n" + coverage_stream.getvalue())

totals = {}
cov_json_path = REPORTS / "coverage.json"
if cov_json_path.exists():
    totals = json.loads(cov_json_path.read_text(encoding="utf-8")).get("totals", {})
finished = datetime.now(timezone.utc)
total_tests = sum(x["tests"] for x in category_results)
all_passed = all(x["status"] == "PASSED" for x in category_results)
final_status = "PASSED" if all_passed and coverage_ok else "FAILED"
statement_pct = totals.get("percent_covered", "N/A")
covered_branches = totals.get("covered_branches", 0)
num_branches = totals.get("num_branches", 0)
branch_pct = round(covered_branches / num_branches * 100, 2) if num_branches else "N/A"
summary = {
    "project": "ModelLab",
    "started": started.isoformat(),
    "finished": finished.isoformat(),
    "status": final_status,
    "total_test_cases": total_tests,
    "categories": category_results,
    "coverage": totals,
    "branch_coverage_percent": branch_pct,
    "coverage_report": "coverage_html/index.html"
}
(REPORTS / "test_results.json").write_text(json.dumps(summary, indent=2), encoding="utf-8")
(REPORTS / "test_execution.log").write_text("\n\n".join(all_output), encoding="utf-8")
rows = "".join(
    f"<tr><td>{escape(x['category'])}</td><td>{x['tests']}</td>"
    f"<td class='{x['status'].lower()}'>{x['status']}</td>"
    f"<td>{x['failures']}</td><td>{x['errors']}</td></tr>"
    for x in category_results
)
```

Figure 2.3: Generation of Coverage Reports and Calculation of Statement and Branch Coverage.

Report: After collecting execution data, the script generates three coverage outputs: a terminal coverage summary, a detailed HTML report, and a machine-readable JSON report. The statement coverage percentage is retrieved from the JSON summary, while branch coverage is calculated using the number of covered branches divided by the total number of branches.

Section 3 – Coverage Execution and Results

3.1 Execution of Code and Branch Coverage Analysis

The coverage analysis was executed using the consolidated testing script, which automatically runs all five testing categories and generates the corresponding coverage reports.

Execution command: python run_all_tests.py

3.2 Overall Statement and Branch Coverage Results

```
======================================================================
STATEMENT AND BRANCH COVERAGE
======================================================================
Name                Stmts   Miss Branch BrPart  Cover   Missing
----------------------------------------------------------------
backend\server.py     193      5     110     35    97%   12->13, 37->exit, 39->exit, 41->exit, 41->exit, 47->exit, 48->exit, 55->56,
 74->exit, 74->exit, 75->exit, 78->exit, 84->exit, 894->exit, 102->exit, 103->exit, 110->exit, 112->a13, 113->114, 113->exit, 119->
exit, 124->exit, 125->exit, 126->exit, 129->exit, 136, 140, 157->exit, 169->exit, 172->exit, 181->exit, 182->exit, 187->exit, 189->
exit, 194->192, 196->exit, 200->201, 204
----------------------------------------------------------------
TOTAL                 193      5    110     35    97%


======================================================================
FINAL RESULT: PASSED
Total tests: 59
Statement coverage: 97.41%
Branch coverage: 68.18%
Dashboard: D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final\reports\index.html
======================================================================
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final>
```

Figure 3.1: Terminal Output Showing Overall Statement and Branch Coverage Results.

Explanation: The terminal output presents the final coverage analysis of the ModelLab backend. A total of 193 statements and 110 branches were identified in backend/server.py. Out of these, 5 statements were not covered during testing. The application achieved 97.41% statement coverage and 68.18% branch coverage, with an overall combined coverage of 86.80%. The overall testing and coverage execution completed successfully with 59 test cases passed.

3.3 Overall HTML Coverage Report

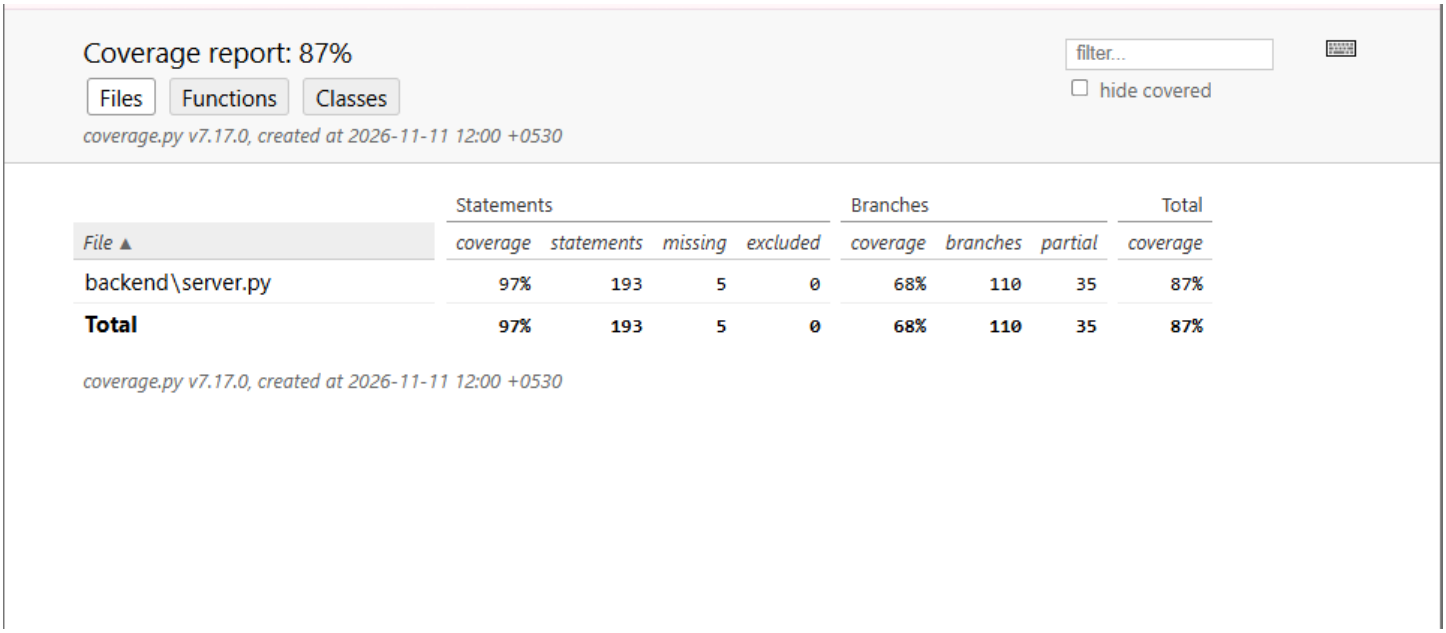
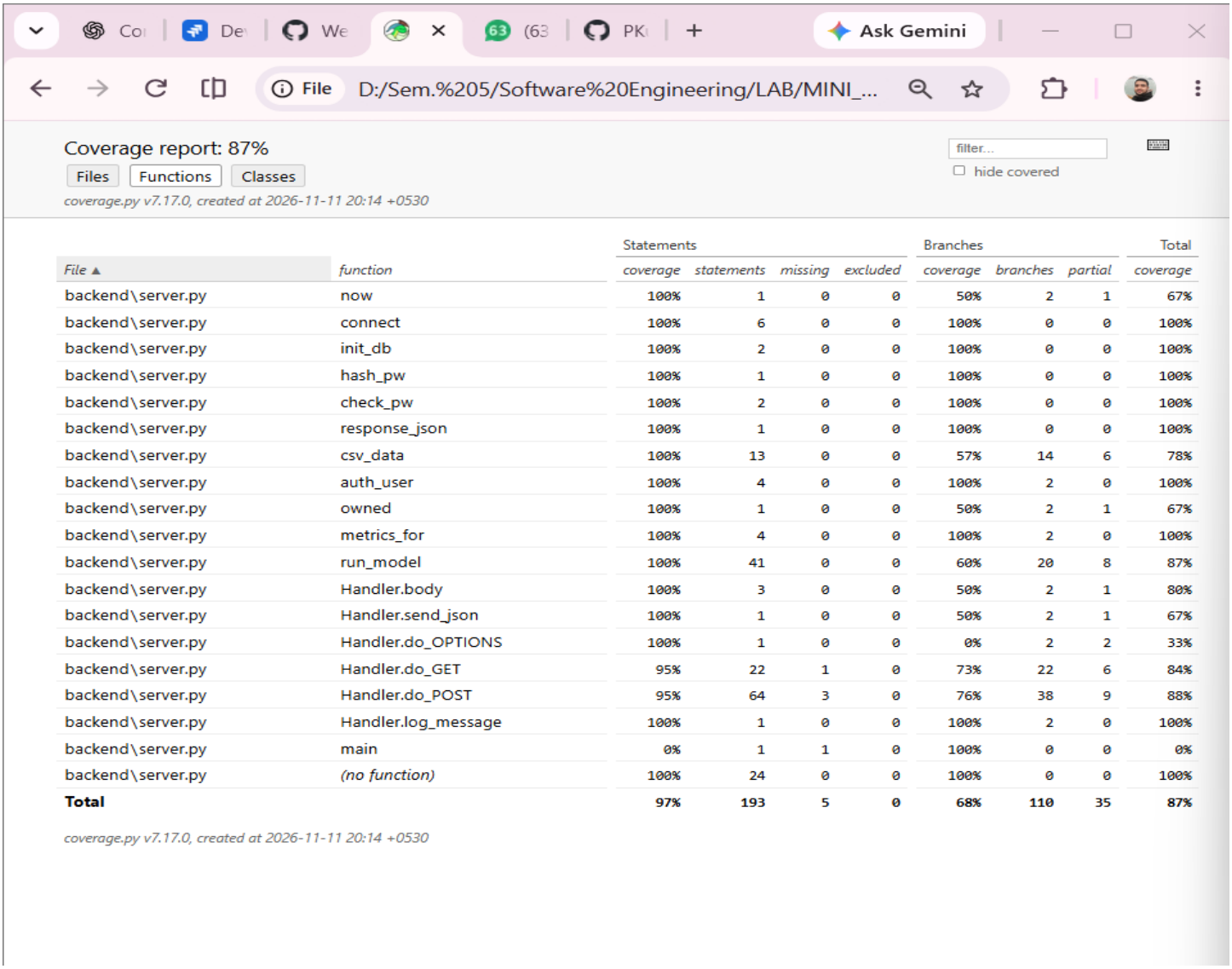


Figure 3.2: Overall HTML Coverage Report Showing Statement and Branch Coverage.
Explanation: The HTML coverage report provides a consolidated view of the measured coverage. It displays statement coverage, the total number of statements, missed statements, branch coverage, total branches, and partially covered branches. The report indicates that the backend achieved approximately 87% overall coverage.

3.4 Function-Level Coverage Analysis



		Statements				Branches			Total
File ▲	function	coverage	statements	missing	excluded	coverage	branches	partial	coverage
backend\server.py	now	100%	1	0	0	50%	2	1	67%
backend\server.py	connect	100%	6	0	0	100%	0	0	100%
backend\server.py	init_db	100%	2	0	0	100%	0	0	100%
backend\server.py	hash_pw	100%	1	0	0	100%	0	0	100%
backend\server.py	check_pw	100%	2	0	0	100%	0	0	100%
backend\server.py	response_json	100%	1	0	0	100%	0	0	100%
backend\server.py	csv_data	100%	13	0	0	57%	14	6	78%
backend\server.py	auth_user	100%	4	0	0	100%	2	0	100%
backend\server.py	owned	100%	1	0	0	50%	2	1	67%
backend\server.py	metrics_for	100%	4	0	0	100%	2	0	100%
backend\server.py	run_model	100%	41	0	0	60%	20	8	87%
backend\server.py	Handler.body	100%	3	0	0	50%	2	1	80%
backend\server.py	Handler.send_json	100%	1	0	0	50%	2	1	67%
backend\server.py	Handler.do_OPTIONS	100%	1	0	0	0%	2	2	33%
backend\server.py	Handler.do_GET	95%	22	1	0	73%	22	6	84%
backend\server.py	Handler.do_POST	95%	64	3	0	76%	38	9	88%
backend\server.py	Handler.log_message	100%	1	0	0	100%	2	0	100%
backend\server.py	main	0%	1	1	0	100%	0	0	0%
backend\server.py	*(no function)*	100%	24	0	0	100%	0	0	100%
Total		**97%**	**193**	**5**	**0**	**68%**	**110**	**35**	**87%**

coverage.py v7.17.0, created at 2026-11-11 20:14 +0530

Caption: Figure 3.3: Function-Level Statement and Branch Coverage Analysis.
Explanation: The function-level report provides coverage details for individual backend functions and methods. It helps identify which functions have complete statement coverage and where certain conditional execution

paths remain partially covered. Functions such as connect, init_db, hash_pw, and check_pw achieved complete statement and branch coverage, while some other functions retained partially covered branches.

3.5 Class-Level Coverage Analysis

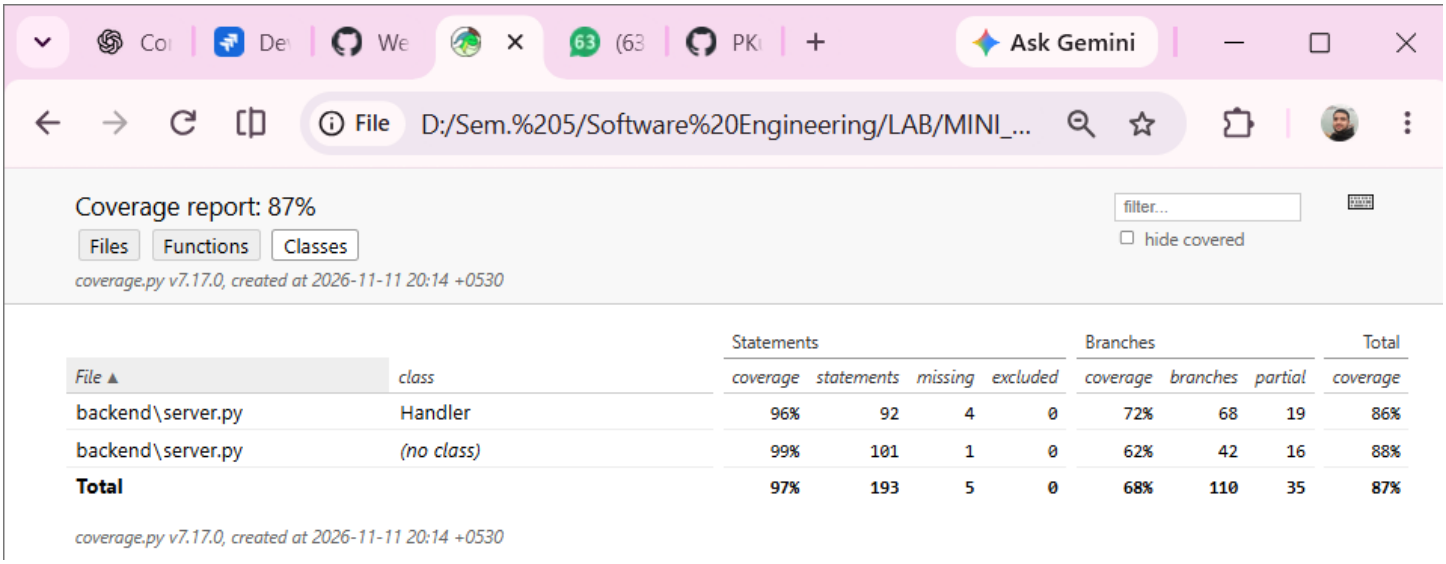


Figure 3.4: Class-Level Coverage Analysis of the ModelLab Backend.

Explanation: The class-level coverage report summarizes statement and branch coverage for the backend classes. The Handler class achieved 96% statement coverage and 72% branch coverage, while the remaining module-level code achieved 99% statement coverage and 62% branch coverage. This analysis helps identify areas requiring additional testing at the class and module levels.

3.6 Consolidated Coverage Results

Include this table in your report:

Coverage metric	Result
Total test cases executed	59
Total statements	193
Executed statements	188
Missed statements	5
Statement coverage	97.41%
Total branches	110
Branch coverage	68.18%
Partially covered branches	35
Combined coverage	86.80%
Displayed combined coverage	87%
Overall execution status	PASSED

Important technical distinction: The report displays 35 partially covered branches (BrPart). This is not the same as saying 35 branches were completely missed. Branch coverage is independently calculated from the covered and total branch counts.

4. ANALYSIS AND CONCLUSION

4.1 Analysis of Statement Coverage

The ModelLab application achieved **97.41% statement coverage**, indicating that the automated testing suite successfully executed most of the statements present in the backend source code.

Out of the 193 statements identified in backend/server.py, 188 statements were executed during testing, while 5 statements remained uncovered. This demonstrates that the implemented test cases exercised the majority of the backend functionality, including authentication, database operations, CSV processing, preprocessing, model execution, and metric calculation.

The achieved statement coverage indicates that the testing suite provides substantial execution coverage of the backend implementation. However, the remaining uncovered statements represent areas that may require additional test scenarios.

4.2 Analysis of Branch Coverage

The application achieved **68.18% branch coverage** across 110 identified branches.
Branch coverage evaluates whether the different possible execution paths of conditional statements have been exercised during testing. Although the testing suite successfully covered a significant proportion of these paths, some conditional branches remained partially covered.

The function-level report indicates that certain functions, including csv_data, owned, and run_model, contain partially covered branches. These results suggest that additional test cases involving alternative conditions, invalid inputs, exceptional scenarios, and boundary conditions could further improve branch coverage. Therefore, branch coverage highlights opportunities to strengthen the testing suite beyond simply executing individual statements.

4.3 Key Findings

The following observations were obtained from the code and branch coverage analysis:

1. **High statement coverage:** The testing suite achieved 86.80% statement coverage, demonstrating that most backend statements were executed.
2. **Partial branch coverage:** The branch coverage of 68.18% indicates that certain conditional execution paths require additional testing.

3. **Function-level analysis:** The HTML coverage report helped identify functions with complete coverage and functions containing partially covered branches.
4. **Class-level analysis:** The coverage report provided a separate assessment of the Handler class and the remaining module-level code.
5. **Automated report generation:** The testing script generated terminal, HTML, and JSON coverage reports, making the results easier to inspect and analyze.
6. **Successful test execution:** All 59 test cases across the five testing categories completed successfully without failures or errors.

4.4 Limitations and Scope for Improvement

Although the achieved coverage demonstrates substantial backend test execution, certain limitations were identified:

- Five backend statements remained uncovered during the execution.
- Branch coverage was lower than statement coverage, indicating that some alternative conditional paths were not fully exercised.
- Additional test cases can be introduced for boundary conditions, exceptional inputs, and alternative execution paths.
- The current coverage measurement focuses on the Python backend, specifically backend/server.py. Browser-side JavaScript and frontend code coverage are not included in this measurement.

Future improvements can focus on increasing branch coverage by designing additional test cases for the partially covered conditions and improving the overall depth of the testing suite.

4.5 Conclusion

The code and branch coverage analysis of ModelLab was successfully completed using the Python coverage.py library integrated into the consolidated testing framework.

The application achieved 97.41% statement coverage and 68.18% branch coverage, with an overall combined coverage of 86.80% (displayed as 87% in the HTML report), and all 59 automated test cases passed successfully. The generated terminal output, HTML dashboard, function-level report, and class-level report provided a detailed understanding of the backend testing coverage.

The analysis demonstrates that the implemented testing suite exercises most of the backend code while also identifying conditional execution paths that can be strengthened through additional testing.

Overall, the coverage analysis helped evaluate the effectiveness of the ModelLab testing suite, identify areas requiring improvement, and establish a measurable basis for enhancing software quality and test completeness.

Approvals

Role	Name	Signature / Email	Date
Project Team Member	**Pratham Kumar** — PES2UG24CS368	prathamkumar041104@gmail.com	11-11-2026
Project Team Member	**Nirupama Jayaraman** — PES2UG24CS324	jnirupama06@gmail.com	11-11-2026
Project Team Member	**Shreyas Bellad** — PES2UG25CS824	shreyasbellad2006@gmail.com	11-11-2026
Project Team Member	**Parinitha Acharya** — PES2UG24CS339	parinitha.acharya2006@gmail.com	11-11-2026
Course Coordinator	**Sheela Devi M.**	sheelam@pes.edu	—

THANK_YOU